



Comp90042

Natural Language Processing

Workshop (Week4 | 2026s1)

Dr Jean Lee





Agenda

1. Discussion (~35 mins)

- **Part of Speech (POS) Tag**
- **Hidden Markov Model (HMM) and Viterbi algorithm**

2. Programming (~25 mins)

23rd Mar	4	 workshop-04.pdf 	05-pos-tagging.ipynb  06-hmm.ipynb  stanford-postagger-4.2.0.jar  english-bidirectional-distsim.tagger 	*Release on end of 27th Mar
----------	---	---	---	--------------------------------



Discussion : Part of Speech (POS) Tag

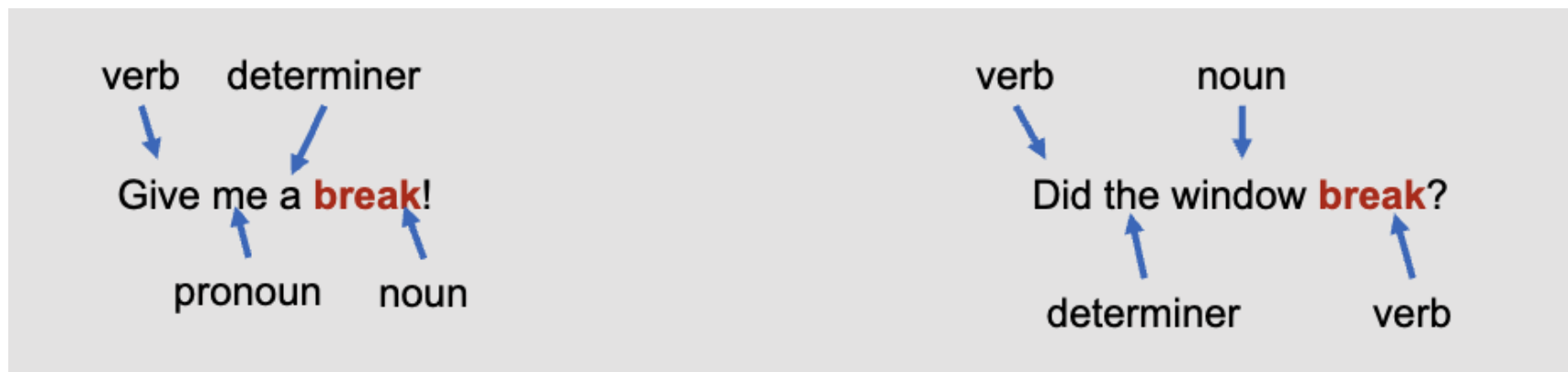
1. What is a **POS tag**?

- (a) POS tag (by hand) the following sentence: Pierre Vinken, 61 years old, will join the board as a nonexecutive director Nov. 29. according to the Penn Treebank tags. (Note that some of the tags are somewhat obscure.)
- (b) What are some common approaches to POS tagging? What aspects of the data might allow us to predict POS tags systematically?

Part of Speech (POS) Tag

What is POS Tag?

- aka word classes
- a **label assigned to word token** in a sentence which indicates some **grammatical properties** (primarily syntactic) of its function in the sentence.
- Token-level classification, Sequence labelling





Part of Speech (POS) Tagsets

Major English tagsets

- Brown
- **Penn Treebank**
- CLAWS/BNC
- “Universal”

Number	Tag	Description
1.	CC	Coordinating conjunction
2.	CD	Cardinal number
3.	DT	Determiner
4.	EX	Existential <i>there</i>
5.	FW	Foreign word
6.	IN	Preposition or subordinating conjunction
7.	JJ	Adjective
8.	JJR	Adjective, comparative
9.	JJS	Adjective, superlative
10.	LS	List item marker
11.	MD	Modal
12.	NN	Noun, singular or mass
13.	NNS	Noun, plural
14.	NNP	Proper noun, singular
15.	NNPS	Proper noun, plural
16.	PDT	Predeterminer
17.	POS	Possessive ending
18.	PRP	Personal pronoun

19.	PRP\$	Possessive pronoun
20.	RB	Adverb
21.	RBR	Adverb, comparative
22.	RBS	Adverb, superlative
23.	RP	Particle
24.	SYM	Symbol
25.	TO	<i>to</i>
26.	UH	Interjection
27.	VB	Verb, base form
28.	VBD	Verb, past tense
29.	VBG	Verb, gerund or present participle
30.	VBN	Verb, past participle
31.	VBP	Verb, non-3rd person singular present
32.	VBZ	Verb, 3rd person singular present
33.	WDT	Wh-determiner
34.	WP	Wh-pronoun
35.	WP\$	Possessive wh-pronoun
36.	WRB	Wh-adverb

POS Tag (by hand)

NNP Pierre
NNP Vinken
 , ,
CD 61
NNS years
JJ old
 , ,
MD will
VB join
DT the
NN board
IN as
DT a
JJ nonexecutive
NN director
NNP Nov.
CD 29
 . .

Number	Tag	Description
1.	CC	Coordinating conjunction
2.	CD	Cardinal number
3.	DT	Determiner
4.	EX	Existential <i>there</i>
5.	FW	Foreign word
6.	IN	Preposition or subordinating conjunction
7.	JJ	Adjective
8.	JJR	Adjective, comparative
9.	JJS	Adjective, superlative
10.	LS	List item marker
11.	MD	Modal
12.	NN	Noun, singular or mass
13.	NNS	Noun, plural
14.	NNP	Proper noun, singular
15.	NNPS	Proper noun, plural
16.	PDT	Predeterminer
17.	POS	Possessive ending
18.	PRP	Personal pronoun

19.	PRP\$	Possessive pronoun
20.	RB	Adverb
21.	RBR	Adverb, comparative
22.	RBS	Adverb, superlative
23.	RP	Particle
24.	SYM	Symbol
25.	TO	<i>to</i>
26.	UH	Interjection
27.	VB	Verb, base form
28.	VBD	Verb, past tense
29.	VBG	Verb, gerund or present participle
30.	VCN	Verb, past participle
31.	VBP	Verb, non-3rd person singular present
32.	VBZ	Verb, 3rd person singular present
33.	WDT	Wh-determiner
34.	WP	Wh-pronoun
35.	WP\$	Possessive wh-pronoun
36.	WRB	Wh-adverb

Note that some of the tags are somewhat obscure!

POS Tag - 05-pos-tagging.ipynb

NNP Pierre
NNP Vinken
,
CD 61
NNS years
JJ old
,
MD will
VB join
DT the
NN board
IN as
DT a
JJ nonexecutive
NN director
NNP Nov.
CD 29
.

```
1 treebank.tagged_sents()[0]
```

```
[('Pierre', 'NNP'),  
 ('Vinken', 'NNP'),  
 (',', ','),  
 ('61', 'CD'),  
 ('years', 'NNS'),  
 ('old', 'JJ'),  
 (',', ','),  
 ('will', 'MD'),  
 ('join', 'VB'),  
 ('the', 'DT'),  
 ('board', 'NN'),  
 ('as', 'IN'),  
 ('a', 'DT'),  
 ('nonexecutive', 'JJ'),  
 ('director', 'NN'),  
 ('Nov.', 'NNP'),  
 ('29', 'CD'),  
 ('.', '.')] ]
```

```
1 treebank.tagged_sents(tagset="universal")[0]
```

```
[('Pierre', 'NOUN'),  
 ('Vinken', 'NOUN'),  
 (',', ','),  
 ('61', 'NUM'),  
 ('years', 'NOUN'),  
 ('old', 'ADJ'),  
 (',', ','),  
 ('will', 'VERB'),  
 ('join', 'VERB'),  
 ('the', 'DET'),  
 ('board', 'NOUN'),  
 ('as', 'ADP'),  
 ('a', 'DET'),  
 ('nonexecutive', 'ADJ'),  
 ('director', 'NOUN'),  
 ('Nov.', 'NOUN'),  
 ('29', 'NUM'),  
 ('.', '.')] ]
```




POS tagging

What are some common approaches to POS tagging?

form
{'NN': 17, 'VB': 1}

- **Unigram :** Assign a POS tag to a token according to **the most common observation in a tagged corpus**; many words are unambiguous.
- **N-gram:** Assign a POS tag to a token according to the most common tag in the **same sequence of n tokens (or tags)** in the tagged corpus; context helps disambiguate.
- **Rule-based:** Write rules that **disambiguate** unigram tags.
- **Sequential:** Learn a **Hidden Markov Model** in a tagged corpus.
- **Classifier-based:** Treat as a **supervised** machine learning problem.



POS tagging

What aspects of the data might allow us to predict POS tags systematically?

- **Frequency**: The most common tags of words can be identified from their frequency in the training corpus (unigram approach).
- **Context**: The context in which a word appears, including the tags of surrounding words (n-gram approach), helps in disambiguating POS tags.
- **Morphology**: Word forms, such as suffixes (e.g., "-ed" for past tense verbs), can inform rule-based and classifier-based tagging.
- **Syntax and Semantics**: The syntactic structure and the semantic relationships between words can be leveraged in rule-based and sequential tagging models to improve accuracy.

POS Tag - 05-pos-tagging.ipynb

worse than unigram due to sparsity

1 unigram_tagger.ta	1 bigram_tagger.tag(exa	1 bigram_tagger.tag(exi	1 stanford_tagger.tag(example_sentence)
<pre>[('You', 'PRP'), ('better', 'JJR'), ('start', 'VB'), ('swimming', None), ('or', 'CC'), ('sink', 'VB'), ('like', 'IN'), ('a', 'DT'), ('stone', 'NN'), (',', ', ', '), ('cause', 'NN'), ('the', 'DT'), ('times', 'NNS'), ('they', 'PRP'), ('are', 'VBP'), ('a', 'DT'), ('-', ':'), ('changing', 'VBG'), ('.', '. ')]</pre>	<pre>[('You', 'PRP'), ('better', None), ('start', None), ('swimming', None), ('or', None), ('sink', None), ('like', None), ('a', None), ('stone', None), (',', ', ', None), ('cause', None), ('the', None), ('times', None), ('they', None), ('are', None), ('a', None), ('-', None), ('changing', None), ('.', '. ', None)]</pre>	<pre>[('You', 'PRP'), ('better', 'JJR'), ('start', 'VB'), ('swimming', 'NN'), ('or', 'CC'), ('sink', 'VB'), ('like', 'IN'), ('a', 'DT'), ('stone', 'NN'), (',', ', ', '), ('cause', 'VB'), ('the', 'DT'), ('times', 'NNS'), ('they', 'PRP'), ('are', 'VBP'), ('a', 'DT'), ('-', ':'), ('changing', 'VBG'), ('.', '. ')]</pre>	<pre>[('You', 'PRP'), ('better', 'RBR'), ('start', 'VB'), ('swimming', 'NN'), ('or', 'CC'), ('sink', 'NN'), ('like', 'IN'), ('a', 'DT'), ('stone', 'NN'), (',', ', ', '), ('cause', 'VB'), ('the', 'DT'), ('times', 'NNS'), ('they', 'PRP'), ('are', 'VBP'), ('a', 'DT'), ('-', 'HYPH'), ('changing', 'NN'), ('.', '. ')]</pre>

by adding backoffs

Hidden Markov Model (HMM)

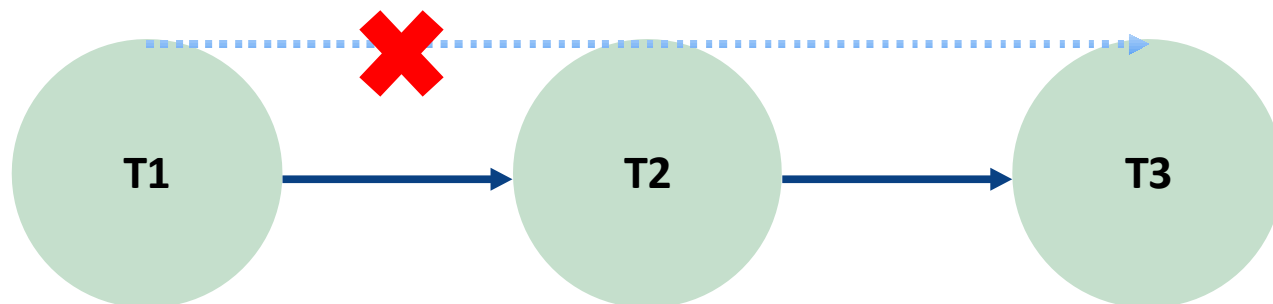
2. What are the assumptions that go into a **Hidden Markov Model**? What is the time complexity of the **Viterbi algorithm**? Is this practical?
 - (a) How can an HMM be used for POS tagging a text? For the purposes of POS tagging:
 - i. How can the initial state probabilities π be estimated?
 - ii. How can the transition probabilities A be estimated?
 - iii. How can the emission probabilities B be estimated?
 - (b) Estimate π , A and B for POS tagging, based on the following corpus:
 1. silver-JJ wheels-NNS turn-VBP
 2. wheels-NNS turn-VBP right-JJ
 3. right-JJ wheels-NNS turn-VBP

Hidden Markov Model (HMM)

What are the assumptions in HMM?

- **Markov assumption:** *the likelihood of transitioning into a given (next) state depends only on the current state, and not the previous state(s) (or output(s))*

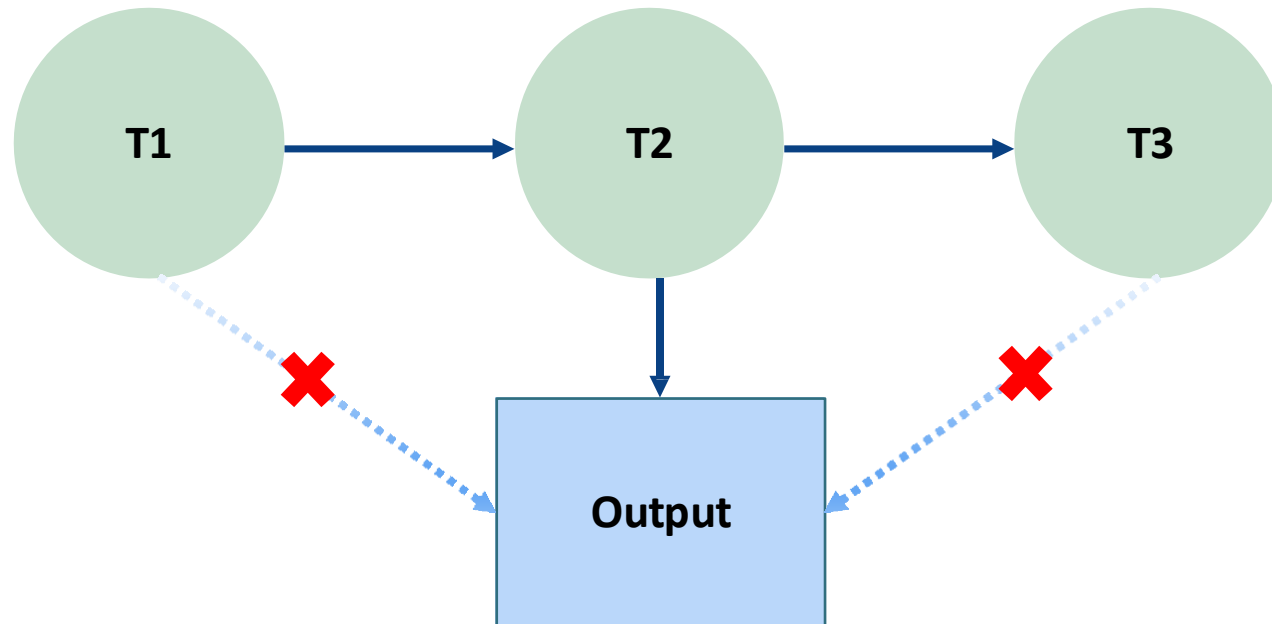
State 3 only depends on state 2, not state 1



Hidden Markov Model (HMM)

What are the assumptions in HMM?

- **Output independence assumption:** *the likelihood of a state producing a certain word (as output) does not depend on the preceding (or following) state(s) (or output(s)).*



Hidden Markov Model (HMM)

How can an HMM be used for POS tagging a text? For the purposes of POS tagging (Sequence labelling)?

$$\hat{t} = \operatorname{argmax}_t P(\mathbf{w} | t) P(t)$$

- **Markov assumption:** (current POS tag | prev. POS tag)

Transition probabilities A (tag-tag pairs)

- The current state (tag) depends only on previous state

(e.g)
$$P(NN | DT) = \frac{\text{count}(DT, NN)}{\text{count}(DT)}$$

$$P(t) = \prod_{i=1}^n P(t_i | t_{i-1}) \quad [\text{Prob. of a tag depends only on the previous tag}]$$

- **Output independence assumption:** (token at i | tag at i)

Emission probabilities B (word-tag pairs)

- An observed event (word) depends only on the hidden state (tag)

$$P(\mathbf{w} | t) = \prod_{i=1}^n P(w_i | t_i) \quad [\text{Prob. of a word depends only on the tag}]$$

(e.g.)
$$P(\text{like} | VB) = \frac{\text{count}(VB, \text{like})}{\text{count}(VB)}$$



Hidden Markov Model (HMM)

What are the parameters do we need to learn when training HMM?

- Initial state probabilities π
 - record the distribution of tags for the **first token** of each sentence
- Transition probabilities A (tag-tag pairs)
 - record the distribution of tags of **the immediately following token**
- Emission probabilities B (word-tag pairs)
 - record the distribution of **corresponding tokens**



Hidden Markov Model (HMM)

Estimate π , A and B for POS tagging, based on the following corpus

1: silver-JJ | wheels-NNS | turn-VBP
2: wheels-NNS | turn-VBP | right-JJ
3: right-JJ | wheels-NNS | turn-VBP

- Initial state probabilities π
 - record the distribution of tags for the **first token** of each sentence
 - In this case, 2 begin with **JJ** and 1 with **NNS**

$$\pi[JJ, NNS, VBP] = [\frac{2}{3}, \frac{1}{3}, 0]$$

Hidden Markov Model (HMM)

Estimate π , A and B for POS tagging, based on the following corpus

1: silver-JJ | wheels-NNS | turn-VBP
2: wheels-NNS | turn-VBP | right-JJ
3: right-JJ | wheels-NNS | turn-VBP

Technically, there are 3 counts of JJ and VBP, but we don't count those that occur at the end of the sentence (it doesn't transition into anything.)

Because if we do that, we'd have transition rows that don't sum up to 1.0 (and they need to sum up to 1.0, so we have a valid probability distribution).

- Transition probabilities A (tag-tag pairs)

- record the distribution of tags of the immediately following token

A	JJ	NNS	VBP
(from) JJ	0	2/2	0
NNS	0	0	3/3
VBP	1/1	0	0

$$P(NN | DT) = \frac{\text{count}(DT, NN)}{\text{count}(DT)}$$

Hidden Markov Model (HMM)

Estimate π , A and B for POS tagging, based on the following corpus

1: silver-JJ | wheels-NNS | turn-VBP
2: wheels-NNS | turn-VBP | right-JJ
3: right-JJ | wheels-NNS | turn-VBP

- Emission probabilities B (word-tag pairs)
 - record the distribution of corresponding tokens

$$P(\text{like} | VB) = \frac{\text{count}(VB, \text{like})}{\text{count}(VB)}$$

B	right	silver	turn	wheels
(from) JJ	2/3	1/3	0	0
NNS	0	0	0	3/3
VBP	0	0	3/3	0

HMM and Viterbi algorithm

3. Consider using the following Hidden Markov Model to tag the sentence `silver wheels turn`:

$\pi[\text{JJ}, \text{NNS}, \text{VBP}] = [0.3, 0.4, 0.3]$ Initial state probabilities π

Transition probabilities A

Emission probabilities B

A	JJ	NNS	VBP	B	silver	wheels	turn
JJ	0.4	0.5	0.1	JJ	0.8	0.1	0.1
NNS	0.1	0.4	0.5	NNS	0.3	0.4	0.3
VBP	0.4	0.5	0.1	VBP	0.1	0.3	0.6

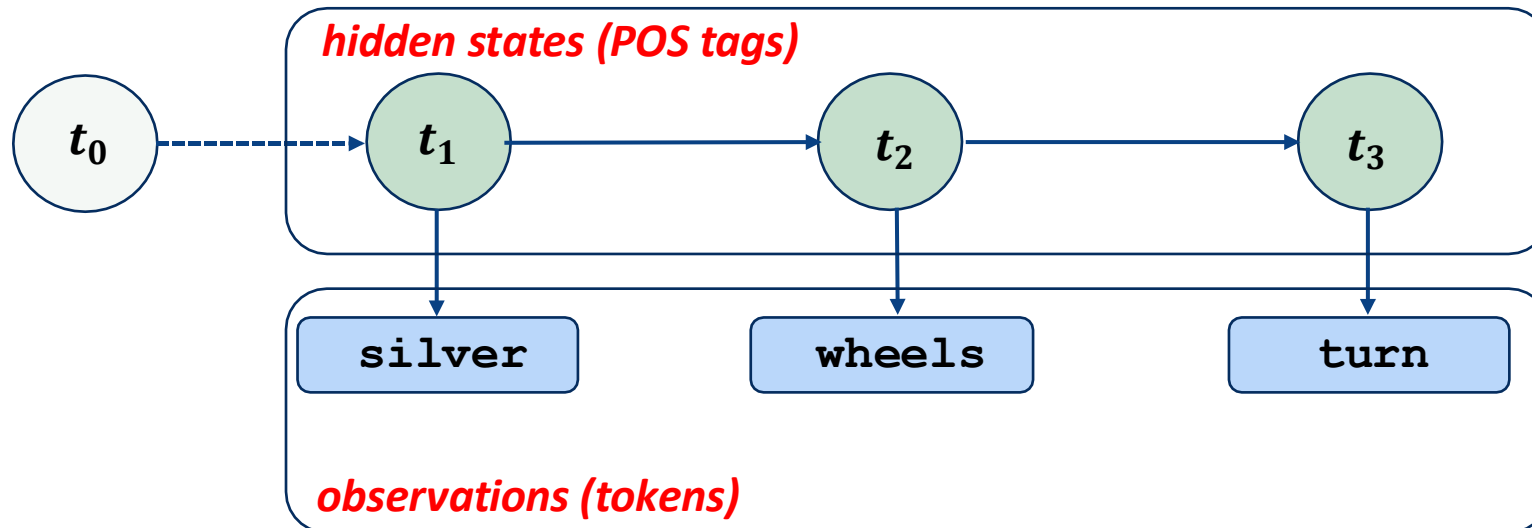
(a) Visualise the HMM as a graph.

(b) Use the **Viterbi algorithm** to find the most likely tag for this sequence.

HMM and Viterbi algorithm

Visualise the HMM as a graph

- Goal: *predicting POS tags when given a sequence of word token*
- POS tags correspond to states of the HMM.
- Tokens correspond to outputs.





HMM and Viterbi algorithm

What is Viterbi algorithm?

- Another **dynamic programming** algorithm
- Most **common decoding** algorithms for HMMs
- *Decoding : the task of determining which sequence of variables (tag) is the underlying source of some sequence of observations (token)*
- Goal : **Compute the joint probability** of the observation sequence together (token) with the best state (tag) sequence.
- **Consider all possible tag combinations, evaluate them, take the max.**



HMM and Viterbi algorithm

What is the time complexity of the Viterbi algorithm? Is this practical?

$O(T^2W)$, where for an HMM with T possible states and a sentence of length W

- In POS tagging,
 - Possible tags: 100
 - Tokens (in a typical sentence): 10
 - Time complexity: $\sim 100,000$
 - Yes, *it's practical*

HMM and Viterbi algorithm

Use the Viterbi algorithm to find the most likely tag for this sequence

$$\pi[\text{JJ}, \text{NNS}, \text{VBP}] = [0.3, 0.4, 0.3]$$

Initial state probabilities π * Emission probabilities B

A	JJ	NNS	VBP	B	silver	wheels	turn
JJ	0.4	0.5	0.1	JJ	0.8	0.1	0.1
NNS	0.1	0.4	0.5	NNS	0.3	0.4	0.3
VBP	0.4	0.5	0.1	VBP	0.1	0.3	0.6

α		1:silver	2:wheels	3:turn
JJ:	JJ	$\pi[\text{JJ}] B[\text{JJ}, \text{silver}]$ $0.3 \times 0.8 = 0.24$		
NNS:	NNS	$\pi[\text{NNS}] B[\text{NNS}, \text{silver}]$ $0.4 \times 0.3 = 0.12$		
VBP:	VBP	$\pi[\text{VBP}] B[\text{VBP}, \text{silver}]$ $0.3 \times 0.1 = 0.03$		

HMM and Viterbi algorithm

Use the Viterbi algorithm to find the most likely tag for this sequence

$$\pi[\text{JJ}, \text{NNS}, \text{VBP}] = [0.3, 0.4, 0.3]$$

A	JJ	NNS	VBP	B	silver	wheels	turn
JJ	0.4	0.5	0.1	JJ	0.8	0.1	0.1
NNS	0.1	0.4	0.5	NNS	0.3	0.4	0.3
VBP	0.4	0.5	0.1	VBP	0.1	0.3	0.6

Recursive step:

compute the probabilities of each tag for the token based on

Transition probabilities A

* Emission probabilities B

Then, choose tag which produce **highest probability**

α	1:silver	2:wheels	3:turn
JJ:	0.24	JJ $\rightarrow$ JJ 0.24 $A[\text{JJ}, \text{JJ}]B[\text{JJ}, \text{wheels}]$ $\times 0.4 \times 0.1 = \mathbf{0.0096}$ NNS $\rightarrow$ JJ 0.12 $A[\text{NNS}, \text{JJ}]B[\text{JJ}, \text{wheels}]$ $\times 0.1 \times 0.1 = 0.0012$ VBP $\rightarrow$ JJ 0.03 $A[\text{VBP}, \text{JJ}]B[\text{JJ}, \text{wheels}]$ $\times 0.4 \times 0.1 = 0.0012$	
NNS:	0.12	JJ $\rightarrow$ NNS 0.24 $A[\text{JJ}, \text{NNS}]B[\text{NNS}, \text{wheels}]$ $\times 0.5 \times 0.4 = \mathbf{0.048}$ NNS $\rightarrow$ NNS 0.12 $A[\text{NNS}, \text{NNS}]B[\text{NNS}, \text{wheels}]$ $\times 0.4 \times 0.4 = 0.0192$ VBP $\rightarrow$ NNS 0.03 $A[\text{VBP}, \text{NNS}]B[\text{NNS}, \text{wheels}]$ $\times 0.5 \times 0.4 = 0.006$	
VBP:	0.03	JJ $\rightarrow$ VBP 0.24 $A[\text{JJ}, \text{VBP}]B[\text{VBP}, \text{wheels}]$ $\times 0.1 \times 0.3 = 0.0072$ NNS $\rightarrow$ VBP 0.12 $A[\text{NNS}, \text{VBP}]B[\text{VBP}, \text{wheels}]$ $\times 0.5 \times 0.3 = \mathbf{0.018}$ VBP $\rightarrow$ VBP 0.03 $A[\text{VBP}, \text{VBP}]B[\text{VBP}, \text{wheels}]$ $\times 0.1 \times 0.3 = 0.0009$	

HMM and Viterbi algorithm

Use the Viterbi algorithm to find the most likely tag for this sequence

$$\pi[\text{JJ}, \text{NNS}, \text{VBP}] = [0.3, 0.4, 0.3]$$

A	JJ	NNS	VBP	B	silver	wheels	turn
JJ	0.4	0.5	0.1	JJ	0.8	0.1	0.1
NNS	0.1	0.4	0.5	NNS	0.3	0.4	0.3
VBP	0.4	0.5	0.1	VBP	0.1	0.3	0.6

Recursive step:

compute the probabilities of each tag for the token based on

Transition probabilities A

* Emission probabilities B

Then, choose tag which produce **highest probability**

α	1:silver	2:wheels	3:turn
JJ:	0.24	0.0096 JJ → JJ	JJ → JJ $A[\text{JJ}, \text{JJ}]B[\text{JJ}, \text{turn}]$ 0.0096 $\times 0.4 \times 0.1 = 0.000384$ NNS → JJ $A[\text{NNS}, \text{JJ}]B[\text{JJ}, \text{turn}]$ 0.048 $\times 0.1 \times 0.1 = 0.00048$ VBP → JJ $A[\text{VBP}, \text{JJ}]B[\text{JJ}, \text{turn}]$ 0.018 $\times 0.4 \times 0.1 = \mathbf{0.00072}$
NNS:	0.12	0.048 JJ → NNS	JJ → NNS $A[\text{JJ}, \text{NNS}]B[\text{NNS}, \text{turn}]$ 0.0096 $\times 0.5 \times 0.3 = 0.00144$ NNS → NNS $A[\text{NNS}, \text{NNS}]B[\text{NNS}, \text{turn}]$ 0.048 $\times 0.4 \times 0.3 = \mathbf{0.00576}$ VBP → NNS $A[\text{VBP}, \text{NNS}]B[\text{NNS}, \text{turn}]$ 0.018 $\times 0.5 \times 0.3 = 0.0027$
VBP:	0.03	0.018 NNS → VBP	JJ → VBP $A[\text{JJ}, \text{VBP}]B[\text{VBP}, \text{turn}]$ 0.0096 $\times 0.1 \times 0.6 = 0.000576$ NNS → VBP $A[\text{NNS}, \text{VBP}]B[\text{VBP}, \text{turn}]$ 0.048 $\times 0.5 \times 0.6 = \mathbf{0.0144}$ VBP → VBP $A[\text{VBP}, \text{VBP}]B[\text{VBP}, \text{turn}]$ 0.018 $\times 0.1 \times 0.6 = 0.00108$

Programming!

Programming

1. In the iPython notebook 05-pos-tagging:

- Why does the bigram tagger — when used without “backoff” — perform worse than the unigram tagger? Find some examples of tokens which are tagged differently by the two models; give evidence from the training corpus as to why they are tagged differently.
- The notebook demonstrates that it helps to use the unigram tagger as a back-off model for the bigram tagger. Why does that mesh with our intuition? Find some examples of tokens that are tagged incorrectly by the unigram model, but correctly by the (backed-off) bigram model.

2. In the iPython notebook 06-hmm:

- The Viterbi algorithm is implemented with loops. Try to implement Viterbi using recursion instead.
- Can you see the difference between the speed of the Viterbi algorithm and the exhaustive search over the lattice? How much faster is Viterbi than exhaustive search on an example problem? (hint: *time* or *clock* functions from the *time* package can be useful)